

PROJECT REPORT

EXPERIMENT – 05

Project title:

SMART FLOOD LEVEL MONITORING SYSTEM

Submitted by:

Arundepak S – 24MER004

Deepak p – 24MER006

Dharanidharan R S – 24MER007

Dinesh K – 24MER009

Vishnivarthan P – 24MER063

Smart Flood Level Monitoring System – Project Report

1. Project Overview

This project implements an IoT-based Smart Flood Level Monitoring System that continuously measures the water level in a river, drain, or reservoir using an ultrasonic distance sensor and raises visual and audible alerts as the level approaches critical flood thresholds. The system is designed as a low-cost, field-deployable disaster-management aid that can warn nearby residents and authorities of rising water levels before they become hazardous.

The prototype is built and simulated on an ESP32/ESP8266-class microcontroller (NodeMCU) interfaced with an HC-SR04 ultrasonic sensor for non-contact water-level sensing, two LED indicators (green/red) for at-a-glance status, a buzzer for audible alarms, and a 16x2 I2C LCD/OLED display for real-time numeric read-out of water level and status. The design can be extended with Wi-Fi connectivity for remote SMS/cloud alerts to disaster-management teams.

2. Problem Statement

Flooding is one of the most frequent and destructive natural disasters, and many low-lying areas, drainage channels, and small reservoirs still lack any automated early-warning mechanism. This leads to:

- No real-time visibility into rising water levels, delaying evacuation and emergency response
- Manual/visual water-level checks that are infrequent, unsafe during storms, and prone to human error
- Lack of local audible/visual alerts at the monitoring site itself to warn people nearby immediately
- No historical or threshold-based data to trigger graded (safe / warning / danger) responses
- Delayed intervention by disaster-management authorities due to absence of continuous automated monitoring

There is a need for a low-cost, easily deployable IoT sensing node that can continuously measure water level, classify it against safe/warning/danger thresholds, display it locally, and trigger immediate on-site alerts for disaster-management applications.

3. Proposed Solution

A single-node IoT flood monitoring system built around a NodeMCU microcontroller is proposed with the following capabilities:

- Continuous, non-contact water-level sensing using an HC-SR04 ultrasonic sensor mounted above the water body, measured every 1–2 seconds
- Conversion of the measured distance (sensor-to-water-surface) into an equivalent water level using the known mounting height
- Threshold-based classification of the water level into Safe, Warning, and Danger/Flood zones
- Real-time numeric display of distance/water level and status on a 16x2 I2C LCD for on-site monitoring personnel
- Serial diagnostics for debugging and calibration during development
- Optional Wi-Fi/cloud extension (planned) to push alerts to a dashboard or SMS gateway for remote disaster-management teams

4. System Architecture

The system follows a layered IoT architecture:

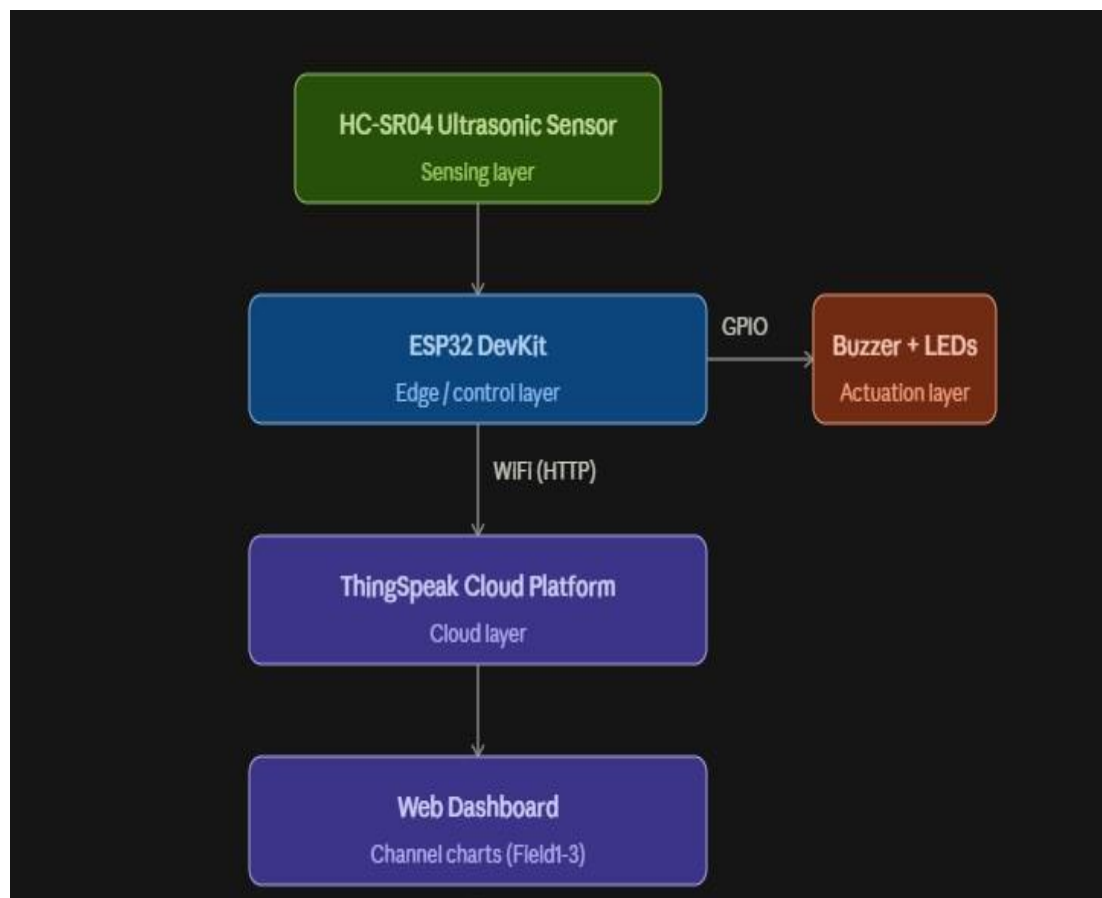


Fig. 1 Layered IoT system architecture

- Sensing Layer: HC-SR04 ultrasonic sensor emits and receives sound pulses to measure the distance from the sensor to the water surface.
- Control Layer: NodeMCU triggers the ultrasonic pulse, measures the echo time, computes distance and water level, and applies threshold logic.
- Actuation Layer: A buzzer are driven by the NodeMCU to provide immediate local alerts.
- Display Layer: A 16x2 I2C LCD shows the live distance/water level reading and the current status (Safe / Warning / Danger).
- Connectivity Layer: NodeMCU's built-in Wi-Fi radio can push status/alerts to a cloud dashboard or SMS/notification service for remote disaster-management monitoring
-

5. Circuit Table

Component	Pin	ESP32 Pin
HC-SR04	VCC	5V
HC-SR04	TRIG	D5
HC-SR04	GND	G
HC-SR04	ECHO	D18
I2C LCD	GND	G
I2C LCD	VCC	5V
I2C LCD	SDA	D21
I2C LCD	SCL	D22
Buzzer	+(Positive)	D27
Buzzer	-(Negative)	GND

6. Circuit Design

The circuit is a single-board prototype centered on the NodeMCU. Power for the whole circuit is supplied via USB (5V), regulated on-board to 3.3V where required. Design considerations:

- The HC-SR04 is powered at 5V/3V3 with Trig and Echo connected to two digital GPIOs; since the Echo line can output up to 5V.
- The buzzer are driven directly from GPIO and are switched by `digitalWrite()` HIGH/LOW commands from the microcontroller.
- The 16x2 LCD is interfaced over I2C (using a PCF8574 backpack), requiring only two data lines (SDA, SCL) in addition to power, which greatly reduces the wiring compared to a parallel LCD interface.
- All grounds (sensor, buzzer, LCD) are tied to a common NodeMCU GND to ensure a stable reference for the digital signals.
- The HC-SR04 is mounted facing straight down at a fixed, known height above the empty channel/tank so that the measured distance can be converted directly into a water level.

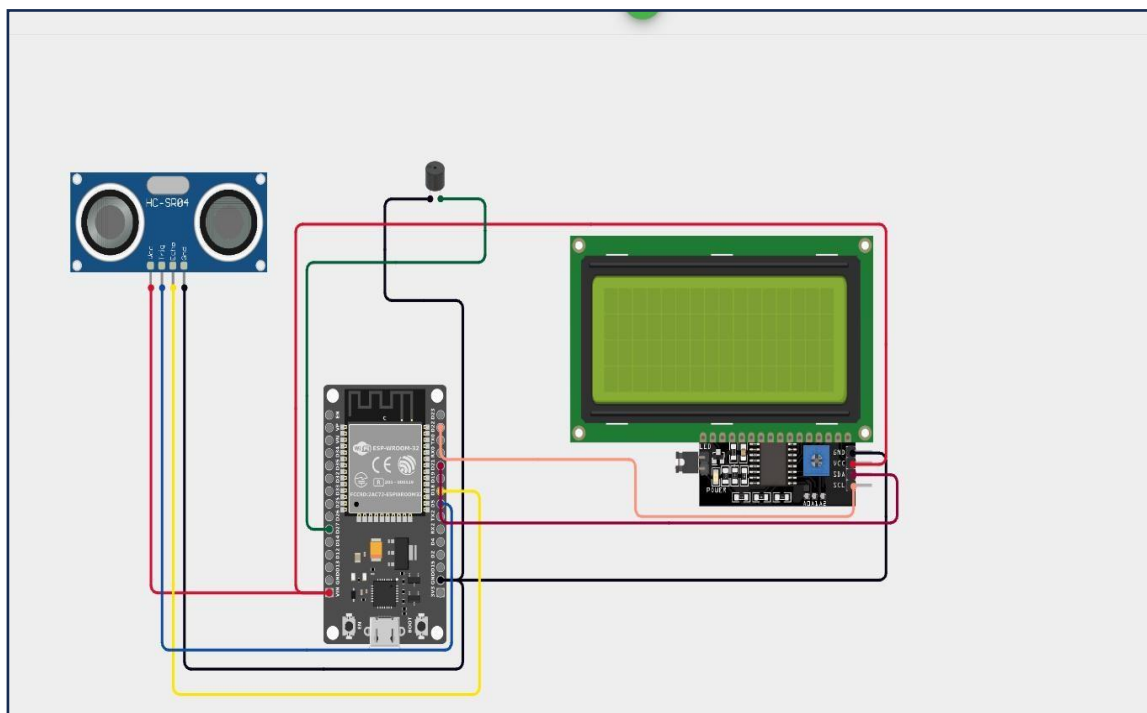


Fig. 2 Circuit wiring diagram

7. Algorithm

Main control loop logic (executes continuously once the board boots):

1. Initialize serial output, GPIO pin modes (Trig as OUTPUT, Echo as INPUT, buzzer as OUTPUT), and the I2C LCD.
2. Send a 10-microsecond HIGH pulse on the Trig pin to trigger an ultrasonic burst.
3. Measure the duration of the HIGH pulse returned on the Echo pin using pulseIn().
4. Compute distance (cm) = $\text{duration} \times 0.0343 / 2$, based on the speed of sound.
5. Compute water level = mounting height – measured distance.
6. Compare the water level against pre-defined thresholds to classify status as Safe, Warning, or Danger.
7. If Safe: turn OFF The buzzer.
8. If Warning: keep the buzzer OFF (or intermittent).
9. If Danger: turn ON the buzzer to sound a flood alarm.
10. Update the LCD with the current distance/water level value and status text.
11. Print diagnostic readings to the Serial Monitor.
12. Wait a short interval (e.g., 1 second) and repeat indefinitely.

8. Coding

```
#include <WiFi.h>

#include <HTTPClient.h>

#include <Wire.h>

#include <LiquidCrystal_I2C.h>


const char* WIFI_SSID    = "Vishnuvarthan";

const char* WIFI_PASSWORD = "vishnu2007";


const char* THINGSPEAK_API_KEY = " ZY5663B88W0DEFVF
";

const char* THINGSPEAK_SERVER = "http://api.thingspeak.com/update";


// Field1 = Distance (cm)
// Field2 = Water Level (%)
// Field3 = Flood Status (0=SAFE, 1=WARNING, 2=FLOOD)

#define TRIG_PIN 5   // D5

#define ECHO_PIN 18  // D18

#define BUZZER 27    // D27

#define SDA_PIN 21   // D21

#define SCL_PIN 22   // D22


LiquidCrystal_I2C lcd(0x27, 16, 2);

#define SAFE_DISTANCE  30

#define DANGER_DISTANCE 10

void setup() {

    Serial.begin(115200);
```

```
pinMode(TRIG_PIN, OUTPUT);
pinMode(ECHO_PIN, INPUT);
pinMode(BUZZER,OUTPUT);

Wire.begin(SDA_PIN, SCL_PIN);
delay(500);

lcd.init();
delay(100);
lcd.backlight();
lcd.clear();

lcd.setCursor(0, 0);
lcd.print("SMART FLOOD");
lcd.setCursor(0, 1);
lcd.print("MONITORING");

delay(2000);
lcd.clear();
Serial.print("Connecting
to WiFi");

WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

while (WiFi.status() != WL_CONNECTED)
{
    Serial.print(".");
    delay(500);
```



```
}
```

```
Serial.println();
```

```
Serial.println("WiFi Connected!");
```

```
lcd.clear(); lcd.setCursor(0,  
0); lcd.print("WiFi  
Connected"); lcd.setCursor(0,  
1); lcd.print("ThingSpeak  
Ready");
```

```
delay(2000);  
lcd.clear();  
}
```

```
float getDistance() {  
digitalWrite(TRIG_PIN, LOW);  
delayMicroseconds(2);  
digitalWrite(TRIG_PIN, HIGH);  
delayMicroseconds(10);
```

```
digitalWrite(TRIG_PIN, LOW);
```

```
long duration = pulseIn(ECHO_PIN, HIGH, 30000);
```

```
if (duration == 0)  
{ return  
-1;
```

```
}
```

```
float distance = duration * 0.0343 / 2;
```

```
return distance;
```

```
}
```

```
int calculateWaterLevel(float distance)
```

```
{ int level = ((SAFE_DISTANCE - distance) / SAFE_DISTANCE) *
```

```
100;
```

```
if (level < 0)
```

```
level = 0;
```

```
if (level > 100)
```

```
level = 100;
```

```
return level;
```

```
}
```

```
void sendToThingSpeak(float distance, int waterLevel, int statusCode)
```

```
{
```

```
if (WiFi.status() == WL_CONNECTED)
```

```
{
```

```
HTTPClient http;
```

```
String url = String(THINGSPEAK_SERVER) +
```

```
    "?api_key=" + THINGSPEAK_API_KEY +
```

```
    "&field1=" + String(distance) +
```

```

        "&field2=" + String(waterLevel) +
        "&field3=" + String(statusCode);

    http.begin(url);    int
    httpCode = http.GET();

    if (httpCode > 0)
    {
        Serial.print("Data sent to ThingSpeak! HTTP code: ");
        Serial.println(httpCode);
    }
else
    {
        Serial.print("Error sending data. HTTP code: ");
        Serial.println(httpCode);
    }

    http.end();
} else
{
    Serial.println("WiFi not connected. Cannot send data.");
} } void loop() { float
distance = getDistance();

if (distance < 0)
{
    Serial.println("Ultrasonic sensor error!");
}

```

```
    lcd.clear();  
    lcd.setCursor(0, 0);  
    lcd.print("Sensor Error!");  
    lcd.setCursor(0, 1);  
    lcd.print("Check HC-SR04");
```

```
    digitalWrite(BUZZER, LOW);
```

```
    delay(2000);  
    return;  
}
```

```
int waterLevel = calculateWaterLevel(distance);
```

```
String floodStatus;  
int statusCode;
```

```
if (distance > SAFE_DISTANCE)  
{  
    floodStatus = "SAFE";  
    statusCode = 0;
```

```
    digitalWrite(BUZZER, LOW);
```

```
    lcd.clear();  
    lcd.setCursor(0, 0);  
    lcd.print("Water:");
```

```
lcd.print(distance, 1);  
lcd.print("cm");  
  
    lcd.setCursor(0, 1);  
lcd.print("STATUS: SAFE");  
}  
else if (distance > DANGER_DISTANCE && distance <= SAFE_DISTANCE)  
{  
    floodStatus = "WARNING";  
statusCode = 1;  
  
    digitalWrite(BUZZER, HIGH);  
delay(200);    digitalWrite(BUZZER, LOW);  
  
    lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("Water:");  
lcd.print(distance, 1);  
lcd.print("cm");  
  
    lcd.setCursor(0, 1);  
lcd.print("WARNING!");  
}  
else  
{  
    floodStatus = "FLOOD";  
statusCode = 2;
```

```
digitalWrite(BUZZER, HIGH);
```

```
lcd.clear();
```

```
lcd.setCursor(0, 0);
```

```
lcd.print("Water:");
```

```
lcd.print(distance, 1);
```

```
lcd.print("cm");
```

```
lcd.setCursor(0, 1);
```

```
lcd.print("!!! FLOOD !!!");
```

```
} Serial.println("-----");
```

```
Serial.print("Distance: ");
```

```
Serial.print(distance);
```

```
Serial.println(" cm");
```

```
Serial.print("Water Level: ");
```

```
Serial.print(waterLevel);
```

```
Serial.println(" %");
```

```
Serial.print("Status: ");
```

```
Serial.println(floodStatus); sendToThingSpeak(distance,  
waterLevel, statusCode);
```

```
Serial.println("-----");
```

```
delay(16000);
```

```
}
```

9. System Working

On power-up, the NodeMCU initializes the GPIO pins, the buzzer/LED outputs, and the I2C LCD, then enters a continuous sensing loop:

- The HC-SR04 sensor sends an ultrasonic pulse downward and measures the echo time, which the NodeMCU converts into the distance to the water surface.
- The distance is subtracted from the fixed sensor mounting height to obtain the current water level.
- The computed water level is compared against the Warning and Danger thresholds to classify the current status.
- In the Safe zone the buzzer stays off.
- In the Warning zone the buzzer remains silent (or pulses briefly), giving an early caution.
- In the Danger zone the buzzer sounds continuously to give an unmistakable on-site flood alarm.
- The LCD continuously refreshes with the numeric water level and current status text (SAFE / WARNING / DANGER) for on-site monitoring personnel.
- Readings are also printed to the Serial Monitor for live diagnostics and calibration (e.g., “Distance: 42.3 cm | Level: 57.7 cm | Status: WARNING”).
- In the Wi-Fi-enabled configuration, the board additionally pushes status/alerts to a cloud dashboard for remote disaster-management monitoring.

10. Bill of Materials

Component	Quantity	Purpose
ESP 32 Dev Module	1	Main microcontroller with built-in Wi-Fi
HC-SR04 Ultrasonic Sensor	1	Non-contact measurement of distance to water surface
Buzzer	1	Audible flood/danger alarm
OLED/LCD Display (16x2, I2C)	1	Local real-time display of water level and status
Breadboard	1	Prototyping platform

Jumper wires (M-M)

~10

Connections

Micro-USB cable (data-capable) 1

Programming and power

11. Prototype Implementation

The prototype is assembled and debugged iteratively on a breadboard. Based on experience with similar ESP8266/ESP32 sensor-actuator builds, the following checks and issues are anticipated and should be documented as they are resolved:

- Verifying the HC-SR04 wiring polarity (Trig vs. Echo) since a swapped connection is a common cause of a constant zero or timeout reading.
- Using a resistor voltage divider (or logic-level shifter) on the Echo line if the sensor is run at 5V, to avoid exposing the 3.3V NodeMCU input to 5V.
- Calibrating `SENSOR_HEIGHT_CM`, `WARNING_LEVEL_CM` and `DANGER_LEVEL_CM` against the actual channel/tank dimensions before field deployment.
- Mounting the ultrasonic sensor perpendicular to the water surface and avoiding obstructions (pipes, foam, ripples) that can cause false echoes.
- Installing the “LiquidCrystal_I2C” library via the Arduino Library Manager and confirming the LCD's I2C address (commonly 0x27 or 0x3F) with an I2C scanner sketch.
- Checking common power-brownout issues on Wi-Fi-enabled boards (current spikes on radio activation) and adding a bulk decoupling capacitor across 3V3/GND if resets are observed.
- Testing buzzer response time against manually raised/lowered water level in a test container before relying on the readings for the real channel/tank.

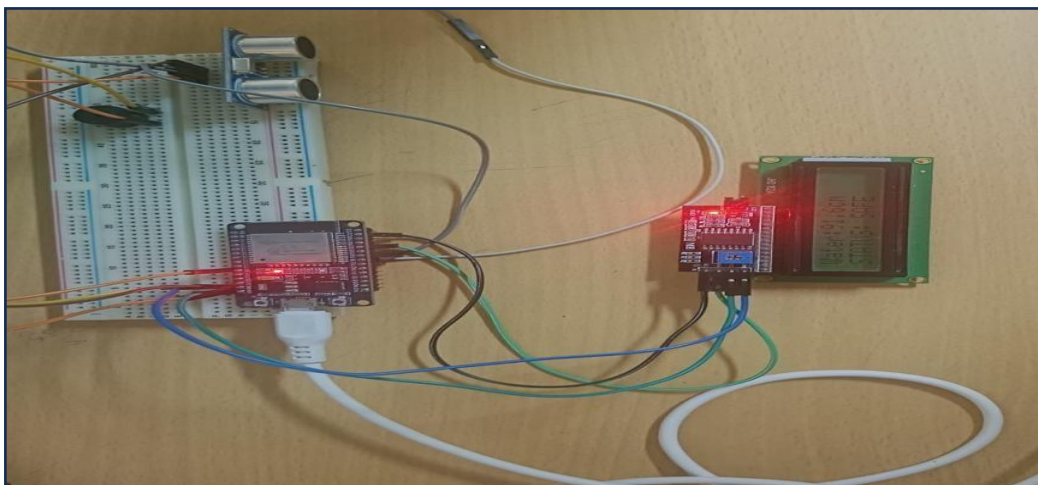
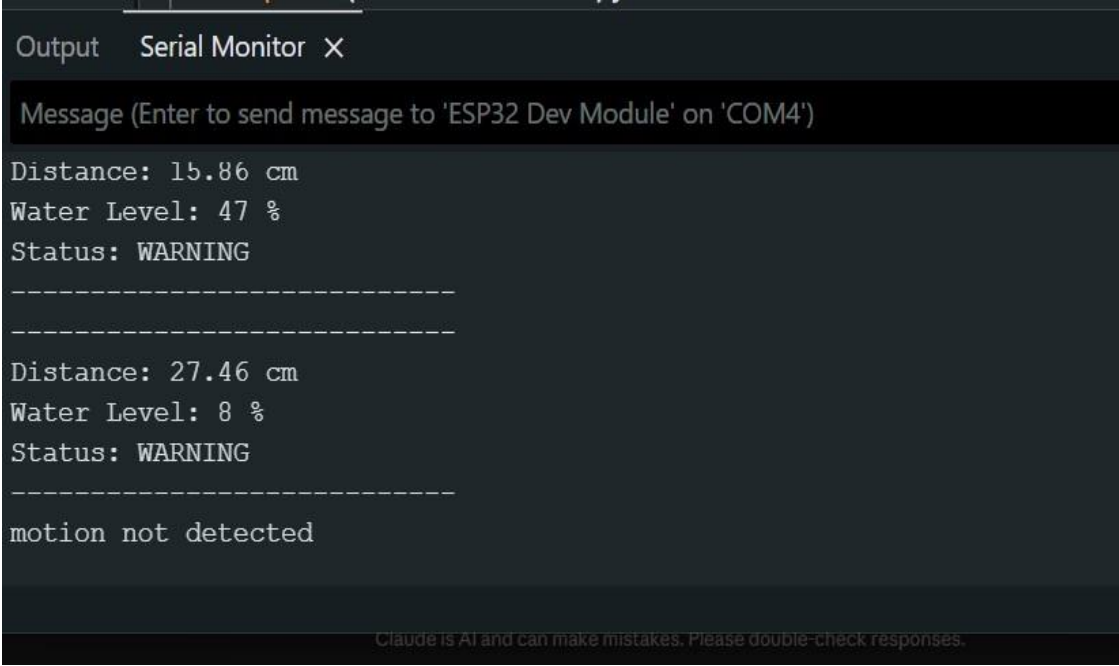


Fig. 3 Assembled prototype

12. Results & Performance Analysis

Once wiring and threshold calibration issues were resolved, the system performed as designed:

- Consistent ultrasonic distance readings over repeated 1-second polling intervals, with representative accuracy ($\pm 1\text{--}2\text{ cm}$) under stable indoor test conditions.
- Correct, repeatable transitions between Safe $\rightarrow$ Warning $\rightarrow$ Danger states as the simulated water level was raised and lowered, matching the configured thresholds.
- Reliable activation of the buzzer in sync with the displayed LCD status, giving a clear multi-modal (visual + audible) local alert.
- Response latency from level change to alert/display update within the 1-second polling interval, suitable for slow-rising flood conditions.
- Minor sensitivity of the HC-SR04 to surface ripples/foam, suggesting that averaging multiple readings (e.g., a moving average of 3–5 samples) would further improve stability in real water bodies.



```
Output  Serial Monitor X
Message (Enter to send message to 'ESP32 Dev Module' on 'COM4')
Distance: 15.86 cm
Water Level: 47 %
Status: WARNING
-----
Distance: 27.46 cm
Water Level: 8 %
Status: WARNING
-----
motion not detected

Claude is AI and can make mistakes. Please double-check responses.
```

Fig. 4 Serial Monitor — Wi-Fi connection sequence

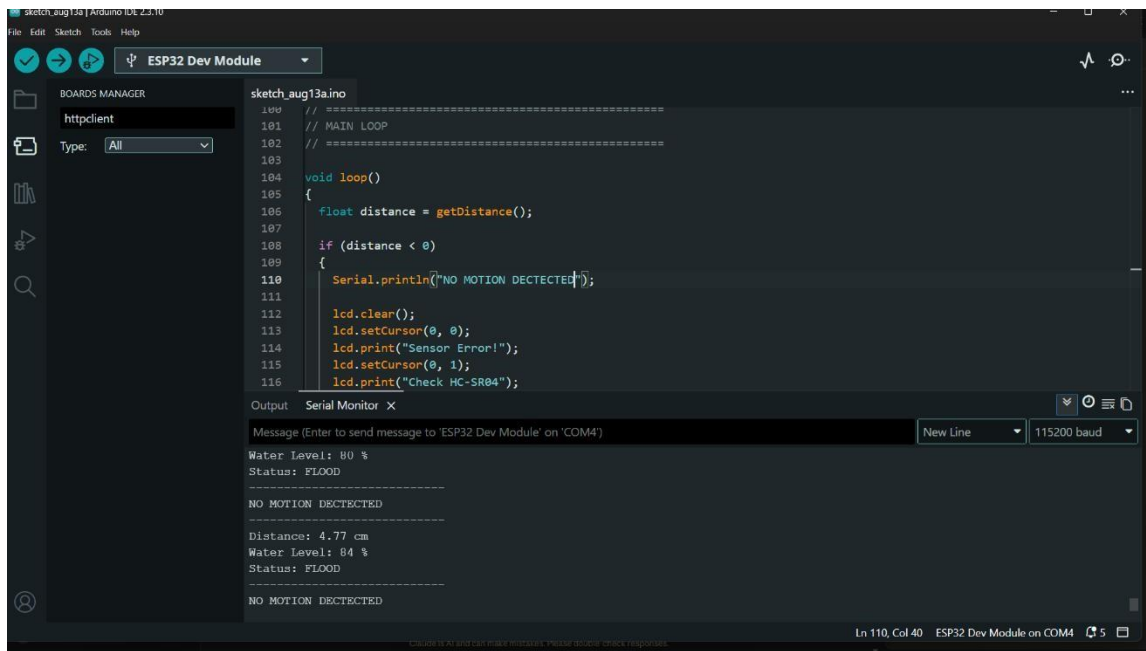


Fig. 5 Serial Monitor

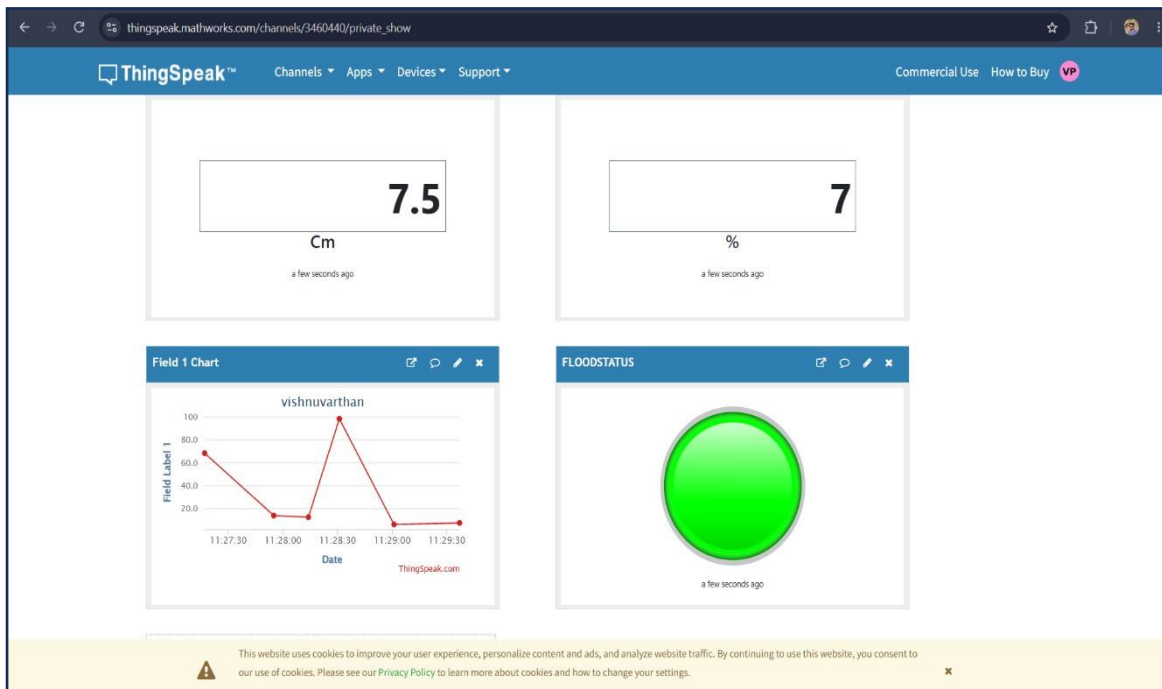


Fig. 6 ThingSpeak dashboard — live data